A Formal Theory of the Cognitive Substrate for Coding Agents

Unifying the substrate faculties, the end-to-end reliability framework, and the forgekit implementation — with a two-layer duality theorem, a unified algorithm set, and a Qur'anic epistemology.

A synthesis edition. Consolidates three independently-developed bodies of work: the *Cognitive Substrate for Coding Agents* (faculties and mechanisms M1–M6, with two runnable prototypes), the *End-to-End Agent Reliability Framework* (failure modes F1/F2, invariants I1–I4, algorithms A1–A7, theorems T1–T6), and the *forgekit / claude-e2e-kit* implementation (deterministic hooks, skills, and committed-file memory). The fourteen-mapping Qur'anic lens is carried throughout as the governing epistemology of obligation.

Abstract

A large language model used for coding is a fixed probabilistic map, $y = f_\theta(x)$: stateless, frozen, and bounded in context. Three research efforts converged, independently, on the same conclusion — that the remedy is not a better prompt or a bigger model but an *external, stateful architecture* wrapped around the frozen core. This paper proves they are describing one object. We show that the substrate's *impact-awareness* faculty and the framework's *change-closure fixpoint* $\Delta*$ are the same mathematics; that the *assumption gate* and the *amnesia equation* $\texttt{assumption} \approx \texttt{argmax}\ P(\texttt{convention} \mid \texttt{training})$ are the same phenomenon; and that both reduce to a single **two-layer duality**: a *probabilistic* instruction layer that raises the probability $p<1$ of correct behaviour, and a *deterministic* interception layer that guarantees a floor. The central theorem states that neither layer alone can make an agent reliable — a direct formalization of the discipline *never trust the output of a probabilistic engine; earn trust with an external check*. We give definitions, the duality theorem with proof, a unified seven-algorithm task loop, the probabilistic failure model $P(\geq 1\ \text{miss}) = 1 - p^n$, and carry through the six correctness theorems of the

reliability framework. Two prototypes — an impact oracle and a complexity-router/assumption-gate — instantiate the deterministic layer and are evaluated honestly. The forgekit / claude-e2e-kit codebase is the deployed binding. The Qur'anic lens supplies the vocabulary of epistemic obligation (tabayyun, amāna, lā taqfu) that names *why* each safeguard is mandatory rather than optional.

Contents

---

1 The convergence — three roads to one architecture

Three efforts set out from different starting points and arrived at the same building.

The first began with a question about human cognition: when a developer opens a file, they carry memory of what already exists, imagine what an edit will break, and verify their reasoning as they type. A frozen language model

does none of this. The *Cognitive Substrate* work[S] named five faculties the model structurally lacks — memory, learning, imagination, self-correction, and impact-awareness — and argued the remedy is an external architecture that supplies them, together with six operating mechanisms (M1–M6: complexity-routing, an assumption gate, decomposition, goal-anchoring, scope-minimality, and inline verification).

The second began with two concrete failures observed in production coding agents. **F1, partial work:** the agent changes code but not the artifacts that depend on it — docs, tests, changelog, examples. **F2, session amnesia:** a later session lacks the project's goals and conventions, so it fills the gaps with assumptions and the developer burns iterations re-explaining. The *End-to-End Reliability Framework*[E] gave these a formal model: a typed artifact graph, a change-closure fixpoint, four invariants, seven algorithms, and six correctness theorems.

The third is a deployed codebase. *forgekit*[K] (and its Claude-specific precursor *claude-e2e-kit*) implements the same discipline as committed files, deterministic lifecycle hooks, and auto-invoked skills — one configuration that binds the architecture onto Claude Code, Codex, Cursor, Gemini, and Aider alike.

The claim of this paper

These are not three similar ideas. They are one architecture described in three vocabularies. The impact-awareness faculty *is* the change-closure fixpoint. The assumption gate *is* the amnesia equation. The substrate's external structure *is* a two-layer duality — and that duality, which the reliability framework states as a design law, is the theorem the whole thing turns on. What each road saw partially, the union sees whole.

The synthesis also inherits a governing discipline, stated plainly by the practitioner who commissioned this work: *AI output is a mathematically calculated probability; it must never be trusted blindly; for the same prompt it can give a different answer, so use only the capability it is genuinely best at, and earn trust with an external check.* We will see that this sentence is not a slogan but the informal statement of the central theorem — the quantity `(1−p)>0` that forces a deterministic layer to exist.

2 The object of study — the frozen map and its five lacks

Fix the model. Let the coding agent's core be a function

> $y = f_\theta(x)$, θ fixed, x the bounded context window, y the sampled output. (1)

Three properties of this map generate every problem the architecture must solve. To avoid a notation collision with the reliability framework's primitives (§3), we label these *model properties* **P1–P3**:

- **P1 — statelessness.** f_θ has no memory across calls; each invocation sees only the current x. Nothing the agent learned yesterday is present today unless something outside the model re-supplies it.
- **P2 — frozen parameters.** θ does not change from use. The agent cannot learn from an outcome by updating weights; any learning must be external.
- **P3 — bounded, undifferentiated context.** x is finite and flat: a long story and a long program are the same kind of object to it, with no privileged channel for goals versus detail. This is the root of goal-drift and of context saturation.

The five faculties are not wishes; each is the direct consequence of one or more of these properties, and each has an external remedy:

Faculty the model lacks	Forced by	External remedy (this architecture)
Persistent memory	P1	A committed store re-injected each session (§6, A4/A5)
Learning from outcomes	P2	Non-parametric experience store; optional parametric adapters (§6)
Imagination / world-model	P1, P3	A typed artifact graph the agent queries before acting (§3.2)

Self-correction	P3	An external verifier and a deterministic completion gate (§4, A6)
Impact-awareness	P1, P3	The change-closure $\Delta*$ computed on the graph (§3.2, A1)

The critical word is *external*. Because θ is frozen (P2) and context is bounded (P3), none of these can be fixed by prompting harder or by fine-tuning alone. The architecture must live *around* the model, hold state *outside* it, and enforce behaviour the model cannot be relied upon to produce on its own. The rest of this paper makes "cannot be relied upon" precise and shows what "enforce" must therefore mean.

---

3 Definitions

3.1 The stateful substrate operator

The architecture turns the stateless map (1) into a stateful operator by threading an external memory M through it.

Definition 1 *(Cognitive substrate)*

A *cognitive substrate* over a frozen model f_θ is an operator

$$(y_t, M_{t+1}) = F(x_t, M_t; f_\theta) \tag{2}$$

where M_t is a persistent store that survives between calls, x_t is the request at step t, and y_t is the output. The store carries what P1–P3 deny the model: prior state, learned priors, the world-model, and the reflection log. F is required to *read* M_t into the model's context before sampling and to *write back* M_{t+1} after — the closed loop the bare model (1) does not have.

Three environmental capabilities are needed to realize M and the read/write loop. The reliability framework calls these its *primitives*. It labels them P1/P2/

P3, which collides with the model properties of §2; we therefore rename them Π_1, Π_2, Π_3 and use that notation for the remainder of the paper.

Primitive	Definition	forgekit binding	Generic binding
Π_1 — persistent store	Files the agent can read/write that survive sessions and travel with the project	git repo: `CLAUDE.md`, `docs/*.md`, `.claude/**`	any VCS; any repo-reading agent
Π_2 — lifecycle interception	Deterministic code executed at fixed points of the agent loop (start, end-of-turn)	hooks: `SessionStart`, `Stop`, `UserPromptSubmit`	pre-commit hooks; CI jobs; IDE tasks
Π_3 — instruction channel	Standing instructions loaded into the model's context	`CLAUDE.md`, `.claude/rules/`, skills	`AGENTS.md`, `.cursorrules`, system prompts

> **The design law, stated early because everything depends on it**
>
> **Π_3 is probabilistic; Π_2 is deterministic.** Instructions (Π_3) raise the probability that the model behaves correctly; interception (Π_2) executes regardless of what the model decides. A reliable substrate needs both, and §4 proves it cannot be built from either alone.

3.2 The repository as a typed artifact graph (the world-model)

The imagination/world-model faculty is realized concretely as a graph over the project's artifacts — the generalization that lets "impact on code" become "impact on *everything that describes or depends on* the code".

Definition 2 *(Typed artifact graph)*

Let the project be a finite set of artifacts $A = \{a_1, \ldots, a_n\}$ with a type function $\tau : A \to \{\texttt{code}, \texttt{test}, \texttt{doc}, \texttt{config}, \texttt{diagram}\}$ and a dependency relation $R \subseteq A \times A$, where $(a,b) \in R$ means "a describes, verifies, exercises, or references b". $R = R_{\text{declared}} \cup R_{\text{discovered}}$: declared edges come from a curated documentation map (high precision, small); discovered edges are found mechanically — a mentions an identifier defined in b — by text search.

The substrate's Prototype I builds this graph from source: an AST parser extracts the code nodes and their edges, so $R_{\text{discovered}}$ over $\{\texttt{code}, \texttt{test}\}$ is computed exactly rather than by grep. The reliability framework's contribution is to widen τ beyond code, making documentation a first-class dependent so that a code change can be seen to obligate a doc change.

Definition 3 *(Dependents operator and change closure)*

For a set $X \subseteq A$, the *dependents* operator is

$$N(X) = \{\, a \in A : \exists\, b \in X, (a,b) \in R \,\} \tag{3}$$

— everything that describes, verifies, or references anything in X. A task seeds a change set $\Delta_0 \subseteq A$ (the files the request names or obviously touches). The *required change closure* is the least fixpoint

$$\Delta_{k+1} = \Delta_k \cup N(\Delta_k),\ \Delta^* = \Delta_k \text{ where } \Delta_{k+1} = \Delta_k. \tag{4}$$

Since A is finite and the sequence is monotone ($\Delta_0 \subseteq \Delta_1 \subseteq \ldots \subseteq A$), the fixpoint exists and is reached in at most $|A|$ steps (Kleene's theorem on a finite lattice). In practice depth 2–3 suffices.

Anchor identity #1: the impact oracle *is* Δ^*

The substrate's Impact Oracle computes a file's blast radius by reverse reachability on the dependency graph — which is exactly the closure (4). The oracle adds a real-valued *confidence* that decays with graph distance,

where the framework's $N(\cdot)$ is boolean; thresholding the oracle's confidence recovers N. They are the same computation. This is why the prototype achieves *perfect recall* on impacted files (§10): reverse reachability, run to fixpoint, cannot miss a reachable dependent.

3.3 The faculties as operators on the store

Each faculty is a well-typed operation on M, and each will be realized by one of the algorithms A1–A7 (§6):

Definition 4 *(Faculty operators)*

- **retrieve :** $(x, M) \to c$ — select the context $c \subseteq M$ relevant to x, favouring entries whose validity has been externally confirmed (validity-anchored memory). *(memory; A5)*
- **impact :** $(\Delta_0, R) \to \Delta*$ — the closure (4). *(impact-awareness; A1)*
- **simulate :** $(\text{edit}, M) \to \text{predicted-effects}$ — consequence estimation over the graph before acting. *(imagination; A1)*
- **verify :** $(y, \text{criteria}) \to \{\text{pass}, \text{fail}, \text{doubts}\}$ — an *external* check, never the generator's self-report. *(self-correction; A3, A6)*
- **route :** $x \to \text{tier}$ and $: \text{fact} \to \text{store-home}$ — transparent classification of effort and of knowledge placement. *(M1; A7)*
- **write-back :** $(M, \text{outcome}) \to M'$ — the bounded-compression checkpoint that makes learning survive P1/P2. *(memory + learning; A4)*

With the object (Def. 1), the world-model (Defs. 2–3), and the faculties (Def. 4) in hand, we can state the result the architecture rests on.

---

4 The central result — the two-layer duality theorem

Everything so far has assumed that "the architecture must enforce behaviour the model cannot be relied upon to produce." We now make "cannot be relied upon" precise and derive what "enforce" must mean.

Definition 5 *(The two layers)*

A substrate's behaviour-shaping is partitioned into two layers over the frozen model:

- The **probabilistic layer** (Π_3): standing instructions loaded into context — `CLAUDE.md`, rules, skills, protocol cards. Let $p = P$(the agent performs the required behaviour on a task — e.g. the full closure Δ^* — under the instruction layer alone).
- The **deterministic layer** (Π_2): code executed at fixed lifecycle points independent of the model's choices — hooks that inject state, or that block a turn. A deterministic check j catches a target miss with probability c_j, and for a decidable structural signal $c_j \to 1$.

Theorem D *(Two-layer duality — neither layer alone suffices)*

Let a task require a behaviour whose omission is a *silent miss*. Under the instruction layer alone the per-task silent-miss probability is $1-p$; under a deterministic layer of k checks the residual silent-miss probability is

$$P(\text{silent miss}) = (1-p) \cdot \prod_{j=1..k} (1-c_j). \tag{5}$$

Then, for any model whose instruction-following is imperfect ($p<1$) and any deterministic layer that is not omniscient ($c_j<1$ for every j):

1. **The probabilistic layer alone cannot reach reliability.** With $k=0$, $P(\text{silent miss})=1-p>0$, and over n tasks $P(\geq 1 \text{ miss})=1-p^n \to 1$. No amount of instruction-writing removes the residual, because instructions are context, not enforcement.
2. **The deterministic layer alone cannot reach reliability either.** A decidable check bounds only the *structural* signal it was built to detect; semantic correctness is undecidable (§12), so $\prod(1-c_j)>0$ for the semantic class. Without the instruction layer raising p, the factor $(1-p)$ stays near 1 and the product is dominated by it.
3. **Their composition is strictly better than either factor.** Because $0<(1-p)<1$ and each $0<(1-c_j)<1$, the product (5) is strictly smaller

than $(1-p)$ and strictly smaller than any single $(1-c_j)$. Reliability is the *product* of a soft factor and hard factors, and needs both kinds present.

□

Proof. Equation (5) is the probability that the behaviour is both omitted by the agent (the independent event of probability $1-p$) and undetected by every one of the k checks (each failing to catch with probability $1-c_j$, taken as conditionally independent given the miss). Claim 1: set $k=0$, the empty product is 1, so $P=1-p$; the n-task bound is the complement of n independent successes, p^n. Claim 2: for the semantic-miss class every decidable c_j is bounded below 1 (Rice's theorem: non-trivial semantic properties of programs are undecidable), so the product cannot vanish; with p not raised, $(1-p)$ is near 1. Claim 3: multiplying a number in $(0,1)$ by further numbers in $(0,1)$ strictly decreases it below every factor. □

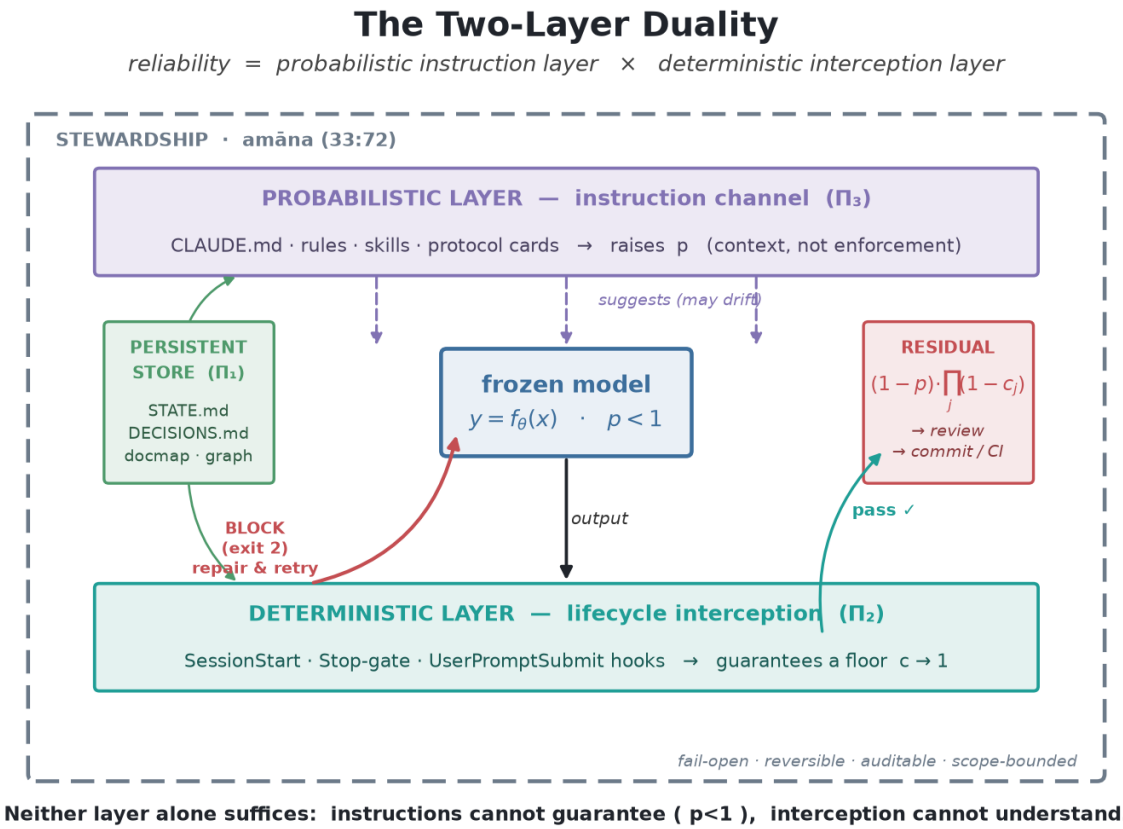

Figure 1. The two-layer duality. The probabilistic instruction layer (Π_3, purple) raises p by loading context but may drift (dashed arrows); the deterministic interception layer (Π_2, teal) executes regardless of the model's choice and either passes the turn or blocks

it (exit 2) back into the model for repair. The persistent store (Π_1) feeds both. What escapes both layers is the residual $(1-p)\cdot\prod(1-c_j)$, handed to review or a later commit/CI gate. The whole sits inside a stewardship boundary (amāna, §9). Neither layer alone suffices — the formal content of the discipline *never trust the output; earn trust with a check*.

What the theorem says in one sentence

The practitioner's rule — *never trust the probability engine's output; verify it* — is the statement $(1-p)>0$. Theorem D turns that intuition into a design mandate: because the soft layer can never drive $(1-p)$ to zero, a deterministic layer *must* exist to multiply it down; and because the hard layer can never catch the semantic class, the soft layer must exist to shrink what reaches it. The substrate is two-layered not by taste but by theorem.

---

5 The probabilistic failure model

Theorem D's equation (5) is worth reading as an engineering instrument, because it explains a lived experience and prices every design choice.

5.1 Why "it works, then forgets" is a certainty, not bad luck

With the instruction layer alone, the chance of at least one partial-work incident over n tasks is $1-p^n$. Even an excellent $p=0.9$ gives **65% after 10 tasks and 96% after 30**. The agent that "usually remembers the docs" is, over a project's lifetime, near-certain to forget them at least once. The failure is geometric, so no degree of prompt-polishing escapes it — only a factor $(1-c_j)$ below 1 can bend the curve.

5.2 Why the deterministic gate is worth exactly one factor

Add one gate whose target is the decidable signal "code changed and no doc/state artifact changed." That signal is checkable in microseconds and $c_1 \approx 0.95$. With $p=0.7$, the per-task silent-miss rate falls from 30% to $(1-0.7)\cdot(1-0.95)=1.5\%$ — a twentyfold reduction from a twenty-line hook. Crucially the *class* of surviving misses changes from "forgot the docs entirely" (structural, now caught) to "updated the docs imperfectly" (semantic, handed

to review). The gate does not make the model think; it removes an entire failure mode from the model's shoulders.

5.3 The lattice of gates

The same classifier can run at three lifecycle points, and (5) shows their catches multiply: a turn-level Stop hook $\subset$ a commit-level pre-commit hook $\subset$ a PR-level CI job. Each later gate catches what earlier ones missed — the product (5) with $k=3$. This is also the answer to portability: where hooks are unavailable, the *same* deterministic check re-binds as a pre-commit hook or a CI step, moving the enforcement point without changing the mathematics.

> **The honest cost side**
>
> The model also prices the developer's real pain. Each silent miss costs a rework loop of expected size $(1-p)\cdot(1+r)$, where r is the re-explaining overhead that session-amnesia (F2) inflates. The architecture attacks both factors: gates convert silent misses into *same-session* fixes ($r \to 0$, caught before the developer sees the result), and persistence makes any residual loop cheap because the context is already standing. This is where the theory meets the user's stated grievance — wasted tokens, time, and quality — and answers it with a quantity, not a promise.

---

6 The unified algorithm set — the TASK loop

The faculties of Def. 4 are realized by seven algorithms. They are the reliability framework's A1–A7, recast here as the operations of the substrate: each is a faculty made mechanical, each binds to one lifecycle point, and together they form a single loop whose progress is guaranteed by an explicit worklist and whose floor is guaranteed by a deterministic gate.

A1 — IMPACT-CLOSURE *(impact-awareness · simulate · before any code)*

```
function IMPACT_CLOSURE(request):
    Δ  ← seeds(request)                    # named files + search hits
    W  ← Δ ; frontier ← Δ
```

```
    while frontier ≠ ∅:                      # BFS to the fixpoint Δ* (D
        E ← grep_edges(I(frontier))          #   R_discovered
            ∪ map_edges(frontier)            #   R_declared
            ∪ interface_edges(frontier)      #   callers, overrides, mi
        new ← E \ W ;  W ← W ∪ new ;  frontier ← new
    check_constraints(W, DECISIONS)          # flag conflicts, never ov
    return { (a, τ(a), action(a), risk(a)) : a ∈ W }   # the impact ta
```

Terminates in $\leq |A|$ rounds (Thm. T5); on termination $W \supseteq \Delta^*$ over discoverable+declared edges. This is *simulate* and *impact* of Def. 4, and it is exactly what the Impact Oracle prototype computes (§10).

A2 — EXECUTE-FULL *(world-model · act on the whole closure)*

```
implement every row of the impact table (code AND tests),
    following conventions drawn from the store M (I3);
partial implementation of the table = definitionally incomplete (Def.
```

A3 — SYNC-VERIFY *(self-correction · make every artifact true again)*

```
function SYNC_VERIFY(changed):
    I ← identifiers(diff(changed))
    for a in DocSet ∪ map_artifacts:
        hits ← grep(a, I)
        if hits ≠ ∅:  read(a); update stale content to match code
        else:         record verified-unaffected(a, "no mention of I")
    if structure_changed: update ARCHITECTURE + diagrams + the map its
    if user_facing:       append CHANGELOG
    update STATE ; return {updated[], verified_unaffected[], doubts[]}
```

The *verify* operator: no artifact is ever declared unaffected without the check running — invariant I1's "∨ verified-unaffected" made mechanical. The `doubts[]` channel enforces I3 (surface ambiguity, do not guess).

A4 — HANDOFF *(memory · write-back · session end)*

```
function HANDOFF(K):
    ensure SYNC_VERIFY ran
    σ ← select(K, priority=[goal, next, decisions, gotchas, in_progres
    write STATE ← σ,  |σ| ≤ B lines        # REWRITE (bounded), not ap
    mirror durable decisions → DECISIONS       # append-only log
    if a convention was corrected → update CLAUDE.md / rules   (self-m
    propose commit                              # committing = portabl
```

The bounded-compression checkpoint (`|σ|≤B≈150`) keeps the loader's cost O(B) forever — the snapshot+WAL pattern: STATE is the mutable snapshot, DECISIONS the durable log.

A5 — REHYDRATE *(memory · retrieve · session start)*

```
on SessionStart(startup | resume | clear):
    record baseline b ← git HEAD            # enables the gate's sess
    inject: STATE (≤8KB) + last 10 commits + uncommitted files + DoD r
```

The *retrieve* operator, made deterministic: continuity moves from "the agent may read the file" to "the context always contains it" — the same $\Pi_3 \to \Pi_2$ upgrade as the gate.

A6 — COMPLETION-GATE *(self-correction · the deterministic floor · end of turn)*

```
on Stop(session s):
    C ← (diff base(s)..HEAD) ∪ worktree changes  − internal bookkeepir
    code ← C ∩ CodeClass \ DocClass \ .claude/    # regex-classified
    docs ← C ∩ DocClass
    if code ≠ ∅ ∧ docs = ∅:  exit 2 + repair checklist + set marker
    else:                     exit 0
```

The hard factor c_1 of Theorem D. STATE counts as a doc artifact, so the weakest way to satisfy the gate is to update session state — which is exactly the continuity invariant I2. One check enforces a floor for *both* F1 and F2. Full decision table and safety proofs in §7.

A7 — KNOWLEDGE-ROUTER *(M1 routing · where every fact lives)*

```
route(f):                                          # first match wins (to
    needed every session, stable      → CLAUDE.md            (always l
    relevant only to paths g          → .claude/rules/x  (paths: g; lo
    a procedure / workflow            → skill               (loads on
    specialist's accumulated patterns → subagent memory
    current work status               → STATE.md            (rewritten
    decision + rationale              → DECISIONS.md        (append-or
```

Keeps always-loaded context O(bounded) while total persisted knowledge grows without limit — the mathematical reason the substrate scales. This is the second face of routing: M1's complexity-router chooses a *model tier* by task difficulty; A7 chooses a *storage home* by knowledge type. Both are transparent and deterministic,

and both reject an opaque-LLM classifier for the same reason — it would reintroduce the very `(1−p)` the hard layer exists to remove.

6.1 The loop

The seven compose into one meta-algorithm that fits every task, from a one-line fix to a multi-file feature:

```
TASK(t):  A5 rehydrate → A1 impact-closure → A2 execute →
verify → A3 sync → A4 handoff → A6 gate
```

Formally, iterate until the unsynced set `U = { a ∈ Δ* : ¬updated(a) ∧ ¬verified(a) } = ∅` — a fixpoint loop whose progress is guaranteed by A1's explicit worklist and whose floor is guaranteed by A6. The soft stages (A1–A5, driven by instructions) raise `p`; the gate (A6, deterministic) guarantees the floor; persistence (A4/A5 over Π_1) carries `essential(K)` across the session boundary.

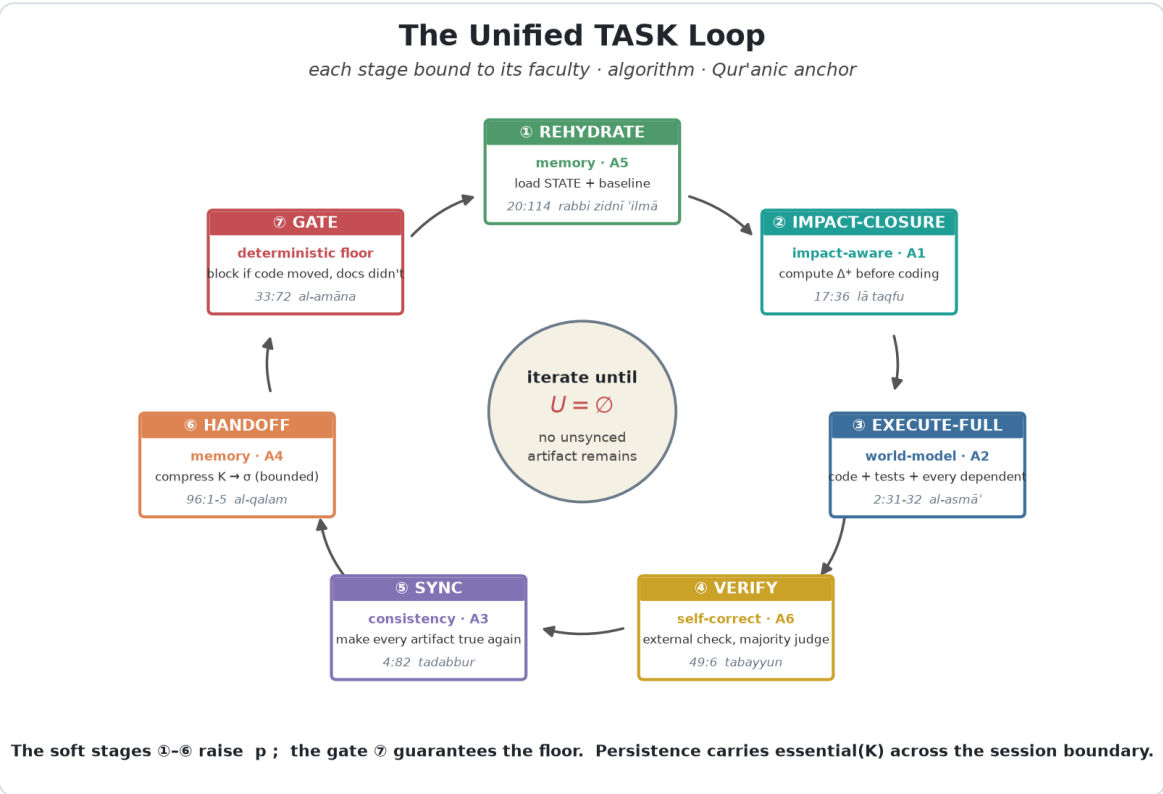

Figure 2. The unified TASK loop. Seven stages, each bound to its faculty, its algorithm (A1–A7), and its Qur'anic anchor (§9). The loop runs to the fixpoint `U=∅` at the hub — no unsynced artifact remains. Soft stages raise `p`; the gate is the

deterministic floor; the store carries state across the boundary. Rehydrate re-arms the baseline that the gate diffs against, closing the cycle across sessions.

---

7 The four invariants and the six correctness theorems

The TASK loop serves four invariants. Stated as logic, they port to any environment that supplies Π_1–Π_3.

Invariant	Statement	Kills
I1 — Consistency	$\forall(a,b)\in R$: changed(b) in task t $\Rightarrow$ updated(a) $\vee$ verified-unaffected(a) in the same t. "No artifact lies about the code."	F1
I2 — Continuity	At every session boundary: essential(K) $\subseteq$ P, and the loader injects it. Corollary: everything essential lives in *committed* files — the only channel that crosses machines, terminals, web, and teammates.	F2
I3 — No fabrication	Every convention the agent acts on is derived from repo evidence, or documented in P, or asked — never sampled from priors. Unattended: assumptions are stated explicitly, never silent.	the amnesia root (§5)
I4 — Verified currency	Facts about the outside world (library versions, APIs) are checked against current sources at use time; the outcome is recorded. Training memory is a stale cache.	stale-knowledge drift

In Hoare-triple form, every task must satisfy

```
{ P consistent ∧ context loaded } execute(t) { Done(t) ∧ P consistent ∧ σ updated }
```

where the gate (A6) checks a decidable necessary condition of the postcondition and the algorithms construct it. **I3 is the invariant the user**

named as the deepest problem — "the biggest problem is assumption." It is the direct architectural answer to the amnesia equation of §5: an under-specified input must be met with supplied context, a stated assumption, or a halt — never a silent guess drawn from the prior.

7.1 The correctness theorems

Six properties are proved of the deterministic layer. They are what make the hard factor of Theorem D trustworthy — a gate that could loop, brick a session, or miss its target signal would not earn its place.

T1 *(No infinite loop; any stop sequence terminates in ≤2 attempts)*

> ***Proof.*** A block sets a per-session marker before exiting 2; marker creation is monotone. Any later attempt matches the marker row ⇒ ALLOW. Independently, the continuation carries `stop_hook_active` ⇒ ALLOW. Two independent guards, either sufficient; even if the process dies between marker-set and exit, the marker is already on disk. □

T2 *(Fail-open: no hook failure can brick a session)*

> ***Proof.*** Every external call is guarded (`|| true`, `2>/dev/null`, a jq→python3→empty fallback chain); every guard-failure path leads to a row whose decision is ALLOW. The only exit-2 path is the deliberate block row. The SessionStart hook only ever exits 0. □

T3 *(Soundness of the block signal — no false silence)*

> ***Proof.*** On the first stop of a session with usable git: `C` is the union of the baseline diff and an untracked-inclusive worktree scan, so every changed path is in `C`; classification is a total function of path; the row order reaches the block row exactly when code changed and no doc changed. Hence a silent code-only completion is impossible at the session's first completion

— the agent must fix the docs or explicitly justify and update STATE, both visible to the developer. □

T4 *(Continuity under handoff)*

Proof. If A4 ran and its commit is pushed/pulled, then for any next session on any machine the loader injects σ at start (A5 reads the committed file), so $\texttt{essential}(K_i) \cap \sigma \subseteq K_{i+1}(\emptyset)$. Residual risk is exactly selection error in the handoff (what it chose not to write), bounded by the priority order and by DECISIONS catching the durable class. □

T5 *(Closure termination, A1)*

Proof. A monotone worklist on the finite set A: each round adds ≥ 1 artifact or stops, so $\leq |A|$ rounds. This is the finite-lattice Kleene fixpoint of Def. 3. □

T6 *(Router totality, A7)*

Proof. The routing chain ends in catch-alls per scope; every fact matches ≥ 1 arm; first-match makes the assignment unique. Hence `route` is a total function — every piece of knowledge has exactly one home, which is what keeps the always-loaded budget bounded. □

How the theorems earn Theorem D

Theorem D says reliability needs a deterministic factor with $c_j \to 1$ on its target signal. T3 is precisely that guarantee (the block fires exactly on the target signal); T1 and T2 ensure the factor is safe to add (it never loops, never bricks); T5 and T6 ensure the soft-layer machinery it composes with is well-defined (the closure terminates, the router is total); T4 extends the

guarantee across the session boundary that F2 attacks. The six local proofs are what make the one global theorem deployable rather than merely true.

---

8 The crosswalk — one object, three vocabularies

The table below is the operational proof of the paper's claim: every concept appears in all three vocabularies, and the final column states the relationship that makes them one object. Three rows (marked ●) are not analogies but identities — the same mathematics under two names. The full machine-readable crosswalk is a companion artifact.

> **Notation reconciliation — the P1/P2/P3 collision**
>
> Both source frameworks independently use the labels P1/P2/P3. In the substrate paper they are model *properties* (P1 statelessness, P2 frozen weights, P3 bounded context, §2); in the reliability framework they are the three *primitives* (persistent store, lifecycle interception, instruction channel). This paper keeps P1–P3 for the model properties and renames the primitives $\mathbf{\Pi_1/\Pi_2/\Pi_3}$ throughout (§3.1). Every reference to a primitive in this paper is written Π_n.

#	Unified concept	Substrate (S)	E2E Framework (E)	forgekit (K)	Relati
1	**The frozen core**	y = f_theta(x): stateless map, fixed weights, bounded context (properties P1,P2,P3)	the agent/model whose behavior instructions can only raise the PROBABILITY of (p<1)	Claude / Codex / Cursor / Gemini / Aider — the model the kit wraps, never modifies	identic the mo probab wrappe
2	**Impact-awareness / partial-work failure ●**	Faculty: impact-awareness gap; the developer silently simulates 'what	F1 partial work; dependents operator N(X); required change closure Delta* =	/impact skill; documentation-map.md (R_declared); grep identifier sweep (R_discovered)	IDENT MATH Oracle reverse depend

		will this edit break'. Prototype I = Impact Oracle (reverse-dependency blast radius with confidence decay)	least fixpoint of X |-> X ∪ N(X); Done predicate; Algorithm A1 IMPACT-CLOSURE		exactly blast-r adds a weight theore boolea
3	**Memory / session-amnesia failure**	Faculty: persistent memory gap; each context window is ephemeral. Validity-anchored memory (facts carry confirmed/ discredited state updated by verified outcomes)	F2 session amnesia; continuity invariant I2 (essential(K_i) ⊆ P and loader L must load it); handoff operator H; K_{i+1} (0)=L(P)	docs/STATE.md (bounded snapshot) + docs/DECISIONS.md (append-only log); / handoff writes it; SessionStart hook injects it	Same f (extern compre Substra anchor a past externa framev snapsh mutabl append
4	**Why assumptions happen (the root the user named) ●**	M2 assumption/ uncertainty gate: under-specified input -> the model confabulates a convention	amnesia equation: when f ∈ essential(K) is missing from L(P), assumption ≈ argmax P(convention | training data) — the mathematically EXPECTED result of missing context, not misbehavior. Invariant I3 (no fabrication)	CLAUDE.md No-assumptions rule; intent-router SPEC card ('state assumptions explicitly')	The fra substra justific argmax fix is to what L missin to hop

5	**Self-correction / verification**	M6 inline verification; Prototype-II verify step; self-correction faculty	Algorithm A6 COMPLETION-GATE (deterministic Stop-hook floor); Hoare postcondition Done(t); the verification operator in A3 (verified-unaffected requires an actual grep, not an assumption)	docs-guard.sh Stop hook (blocks finish if code changed but no doc/state artifact did); reviewer agent verdict	Same v framed framev termin (T3) ar determ duality
6	**Complexity routing**	M1 complexity-aware router (transparent additive rubric); Prototype-II router	Algorithm A7 KNOWLEDGE-ROUTER (where every fact lives, keeps always-loaded context bounded); System 1/System 2 effort routing; intent DFA	intent-router.sh (UserPromptSubmit hook, keyword DFA, <10ms, zero-token); effort-routing rule in CLAUDE.md; per-agent model: fields	Two fa princip routes COMP (cost). by KN storage budget transpa both re classifi
7	**Task decomposition**	M3 task/session decomposition	HTN closure -> ordered task list (A1 output is the worklist); the meta-algorithm TASK(t)	sdlc-pilot 7-phase skill; subagents; git worktrees	Same: into th worklis
8	**Goal-anchoring**	M4 goal-anchoring (goal drift: to a text model a long story and long	BDI Desires = written goal + acceptance criteria in STATE.md; I2 keeps them	docs/STATE.md 'Current goal' + 'Acceptance criteria'; sdlc-pilot SPEC->VERIFY gate	Substra framev persist so ever same t

		code are the same object)	across sessions; acceptance criteria written at SPEC, consumed at VERIFY		
9	**Anti-over-engineering**	M5 anti-over-engineering (scope minimality; the residual whitespace)	amana / scope-boundedness (agent may not exceed asked scope); I3 (no invented structure)	CLAUDE.md effort routing 'trivial -> do it directly, no ceremony'; least-privilege defaults	Weake substra gap, fra a stewa an effo
10	**World-model of the codebase**	Faculty: world-model; Prototype-I codebase world-model (AST -> persistent dependency graph)	the typed artifact graph (A, tau, R = R_declared ∪ R_discovered)	documentation-map.md + the repo itself + ARCHITECTURE.md	Same g it from framew node ty {code,t so DO depend genera 'impact on all a
11	**Continual learning from outcomes**	Faculty: learning without touching theta (non-parametric always-on + parametric LoRA/EWC)	Reflexion loop made cross-session (I3/I4); DECISIONS.md as precedent DB; A4 mirrors durable lessons	Reflexion rule in CLAUDE.md; STATE gotchas; agents' memory: project	Same 'l retrain verbal re-inje parame keeps i parame
12	**Rehydration (session start)**	closed-loop write-back/read-back band in the substrate architecture	Algorithm A5 CONTEXT-REHYDRATE; loader L; records git baseline for the gate	session-context.sh SessionStart hook; / catchup skill (deep variant)	The rea substra made c don't h
13		self-correction faculty; the	LLM-as-Judge applied: reviewer		Substra that se

	Independent verification / judge	honest-negative-result caution (models correct poorly alone)	agent (fresh context, explicit criteria, adversarial); self-consistency for critical changes (majority of N)	reviewer.md agent; DoD item 7	weak; operati EXTER the sar externa the gat
14	**The two-layer duality (THE central new insight) ●**	implicit: the substrate wraps a probabilistic core with deterministic external structure, but v2 never states it as a law	DESIGN LAW: instructions (Pi3) are PROBABILISTIC (raise p); interception (Pi2) is DETERMINISTIC (guarantee a floor c->1). P(silent miss) = (1-p)·prod_j(1-c_j). Since p<1 always, neither layer alone suffices.	the split itself: CLAUDE.md/rules/skills = soft layer; hooks (docs-guard, session-context, intent-router) = hard layer	THIS i substra the sub layers, The us ('never calcula exactly the det
15	**The probabilistic failure model**	eval honesty: perfect accuracy shows separation not a benchmark; p<1	P(>=1 miss) = 1 - p^n over n tasks (0.9 -> 65% at 10, 96% at 30); layered: P(silent miss)=(1-p)·prod(1-c_j)	the lattice of gates: turn-level (hook) ⊂ commit-level (pre-commit) ⊂ PR-level (CI)	The ma works forgets rather quanti determ down t
16	**Stewardship / governance boundary**	STEWARDSHIP/ amana wrapper (33:72) around the whole architecture	amana in I3/I4 as no-fabrication + verified-currency; least privilege, reversibility, logged rationale,	committed-files-only (auditable), block-at-most-once (no nagging), fail-open safety (T2), DOCS_GUARD_DISABLE auditable escape hatch	The et substra is reali proper (fail-op kit (au

			scope-boundedness		

The three anchor identities

1. **Impact-Oracle blast-radius ≡ change-closure Δ*.** Reverse reachability on the dependency graph is the least fixpoint of `X ↦ X ∪ N(X)`. The oracle weights it with confidence decay; the framework's `N(·)` is its boolean core.
2. **M2 assumption gate ≡ the amnesia equation.** `assumption ≈ argmax P(convention | training)` is *why* an under-specified prompt is answered with a confabulated convention — so the gate supplies the missing context or halts; it never hopes.
3. **The substrate's two layers ≡ the design law.** Instructions raise `p<1`; interception guarantees a floor `c→1`. Theorem D. This is the formal statement of the governing discipline.

---

9 The Qur'anic epistemology — the full fourteen mappings

How the lens is used, and how it is not

The Qur'an is used here as a *framing lens and ethics source*, never as technical authority for an engineering claim. No verse proves that an algorithm works or a data structure is correct — those stand or fall on their engineering merits alone (§4–§7). What the lens supplies is (1) a vocabulary for the agent's epistemic obligations — what it owes to truthfulness, verification, and stewardship; (2) a hierarchy of knowledge (ʿilm → fahm → ḥikma) that motivates a *layered* memory rather than a flat store; and (3) ethical constraints on autonomy (amāna, tabayyun) that translate into concrete safeguards. A mapping marked load-bearing directly motivates a specific architectural decision — e.g. a *mandatory* gate, not an optional one; a mapping marked metaphor is illustrative. Even load-

bearing mappings must be independently defensible: the verse explains *why* we insist, not *that* it works.

All Arabic is the clean Uthmani-script edition and all translations are Abdel Haleem, retrieved verbatim via the quran.ai connector; tafsir references are Ibn Kathir. No Qur'anic text is reproduced from model memory.

The fourteen mappings are the epistemology behind the architecture: each names an obligation that a probabilistic engine, left alone, will not honour, and which the deterministic layer therefore exists to enforce. Twelve are load-bearing; two are metaphor. They are grouped by the faculty they govern.

9.1 Impact-awareness — know before you act

Q 17:36 — LĀ TAQFU MĀ LAYSA LAKA BIHI 'ILM LOAD-BEARING

وَلَا تَقْفُ مَا لَيْسَ لَكَ بِهِ عِلْمٌ ۚ إِنَّ السَّمْعَ وَالْبَصَرَ وَالْفُؤَادَ كُلُّ أُولَٰئِكَ كَانَ عَنْهُ مَسْئُولًا

"Do not follow blindly what you do not know to be true: ears, eyes, and heart, you will be questioned about all these."

Before any mutation, the agent must run a pre-action check of what will be affected, whether it has sufficient context, and whether the predicted outcome is evidence-supported rather than pattern-matched. This is *anchor identity #1*: the impact closure Δ* (A1) computed before coding. Ibn Kathir glosses the verse (via Qatadah) as a prohibition on claiming knowledge one lacks — for the agent: do not call a file safe to modify without reading it, nor a test passing without running it. The verse's 'ears, eyes, and heart … questioned' maps to the audit trail: every channel used is logged so the decision can be reconstructed.

9.2 Self-correction — verify, reflect, iterate

Q 49:6 — FA-TABAYYANŪ (THE VERIFICATION GATE) LOAD-BEARING

يَا أَيُّهَا الَّذِينَ آمَنُوا إِن جَاءَكُمْ فَاسِقٌ بِنَبَإٍ فَتَبَيَّنُوا أَن تُصِيبُوا قَوْمًا بِجَهَالَةٍ فَتُصْبِحُوا عَلَىٰ مَا فَعَلْتُمْ نَادِمِينَ

"Believers, if a troublemaker brings you news, check it first, in case you wrong others unwittingly and later regret what you have done,"

The operative term *tabayyun* demands active investigation, not passive acceptance — a mandatory step between receiving information and acting. This is the completion gate A6 and the external verifier of Def. 4: before applying a fix based on an error report, a request, or its own diagnosis, the agent re-reads the file and re-confirms the error. The gate is *architectural* (a deterministic pipeline step), not advisory — the verse's command is categorical, exactly the hard factor c→1 of Theorem D.

Q 4:82 — TADABBUR (SELF-CONSISTENCY) LOAD-BEARING

أَفَلَا يَتَدَبَّرُونَ الْقُرْآنَ ۚ وَلَوْ كَانَ مِنْ عِندِ غَيْرِ اللَّهِ لَوَجَدُوا فِيهِ اخْتِلَافًا كَثِيرًا

"Will they not think about this Quran? If it had been from anyone other than God, they would have found much inconsistency in it."

'They would have found much inconsistency' makes internal contradiction evidence of flawed origin — the argument behind self-consistency checking (SYNC-VERIFY's `doubts[]` channel, and the majority-of-N escalation of the reviewer). If a planned change contradicts the agent's stated reasoning or the tests it just read, the metacognitive pass flags it and halts.

Q 47:24 — TADABBUR ('LOCKS ON HEARTS') METAPHOR

أَفَلَا يَتَدَبَّرُونَ الْقُرْآنَ أَمْ عَلَىٰ قُلُوبٍ أَقْفَالُهَا

"Will they not contemplate the Quran? Do they have locks on their hearts?"

The 'locks on hearts' image maps to a real failure mode: when context saturates, the agent becomes functionally unable to reconsider. The metacognitive controller must be able to reset the working context and re-

examine from a fresh framing — structured backtracking, the fresh-context reviewer that never inherits the builder's narrative.

9.3 World-model — know what already exists

Q 2:31–32 — TA'LĪM AL-ASMĀ' LOAD-BEARING

وَعَلَّمَ آدَمَ الْأَسْمَاءَ كُلَّهَا ثُمَّ عَرَضَهُمْ عَلَى الْمَلَائِكَةِ فَقَالَ أَنبِئُونِي بِأَسْمَاءِ هَٰؤُلَاءِ إِن كُنتُمْ صَادِقِينَ ﴿٣١﴾ قَالُوا سُبْحَانَكَ لَا عِلْمَ لَنَا إِلَّا مَا عَلَّمْتَنَا ۖ إِنَّكَ أَنتَ الْعَلِيمُ الْحَكِيمُ ﴿٣٢﴾

"(2:31) He taught Adam all the names [of things], then He showed them to the angels and said, 'Tell me the names of these if you truly [think you can].' (2:32) They said, 'May You be glorified! We have knowledge only of what You have taught us. You are the All Knowing and All Wise.'"

'He taught Adam all the names' — knowledge begins with naming entities and their relations. This is the typed artifact graph (Defs. 2–3): not a flat file listing but a semantic map of which function calls which, which test covers which class. The angels' 'we have knowledge only of what You have taught us' is precisely the model's situation — it knows only its context window; the external world-model supplies the names of entities that exceed it.

9.4 Continual learning — knowledge as ongoing increase

Q 20:114 — RABBI ZIDNĪ 'ILMĀ LOAD-BEARING

فَتَعَالَى اللَّهُ الْمَلِكُ الْحَقُّ ۗ وَلَا تَعْجَلْ بِالْقُرْآنِ مِن قَبْلِ أَن يُقْضَىٰ إِلَيْكَ وَحْيُهُ ۖ وَقُل رَّبِّ زِدْنِي عِلْمًا

"exalted be God, the one who is truly in control. [Prophet], do not rush to recite before the revelation is fully complete but say, 'Lord, increase me in knowledge!'"

'Do not rush … but say, increase me in knowledge' — two mechanisms at once. The prohibition on rushing before revelation is complete is the halt-on-incomplete-context rule (I3). The prayer for increase is the cross-session

experience store that grows priors without touching θ (P2): learning is external memory that changes what the agent *sees* next, not a weight update.

Q 39:9 — HAL YASTAWĪ METAPHOR

أَمَّنْ هُوَ قَانِتٌ آنَاءَ اللَّيْلِ سَاجِدًا وَقَائِمًا يَحْذَرُ الْآخِرَةَ وَيَرْجُو رَحْمَةَ رَبِّهِ ۗ قُلْ هَلْ يَسْتَوِي الَّذِينَ يَعْلَمُونَ وَالَّذِينَ لَا يَعْلَمُونَ ۗ إِنَّمَا يَتَذَكَّرُ أُولُو الْأَلْبَابِ

"What about someone who worships devoutly during the night, bowing down, standing in prayer, ever mindful of the life to come, hoping for his Lord's mercy? Say, 'How can those who know be equal to those who do not know?' Only those who have understanding will take heed."

'How can those who know be equal to those who do not?' establishes that knowledge is not fungible with ignorance — motivating the architecture's distinction between *grounded* mode (relevant prior experience loaded) and *ungrounded* mode (base model alone), and surfacing that distinction to the user rather than hiding it.

9.5 Persistent memory — externalize to endure

Q 96:1–5 — IQRA' / 'ALLAMA BI-L-QALAM LOAD-BEARING

اقْرَأْ بِاسْمِ رَبِّكَ الَّذِي خَلَقَ ﴿١﴾ خَلَقَ الْإِنسَانَ مِنْ عَلَقٍ ﴿٢﴾ اقْرَأْ وَرَبُّكَ الْأَكْرَمُ ﴿٣﴾ الَّذِي عَلَّمَ بِالْقَلَمِ ﴿٤﴾ عَلَّمَ الْإِنسَانَ مَا لَمْ يَعْلَمْ ﴿٥﴾

"(96:1) Read! In the name of your Lord who created: (96:2) He created man from a clinging form. (96:3) Read! Your Lord is the Most Bountiful One (96:4) who taught by [means of] the pen, (96:5) who taught man what he did not know."

'Who taught by the pen' — al-qalam is the instrument of externalization, turning ephemeral thought into durable record. Every context window is ephemeral (unwritten thought), so the persistent store (Π_1) is 'the pen':

write-back (A4) captures what is learned, retrieval (A5) reloads it next session, consolidation organizes raw experience into structured knowledge. 'Taught man what he did not know' — the pen does not merely record; it grants access beyond unaided capacity.

9.6 Stewardship — the weight of the trust

Q 33:72 — AL-AMĀNA (THE TRUST) LOAD-BEARING

إِنَّا عَرَضْنَا الْأَمَانَةَ عَلَى السَّمَاوَاتِ وَالْأَرْضِ وَالْجِبَالِ فَأَبَيْنَ أَن يَحْمِلْنَهَا وَأَشْفَقْنَ مِنْهَا وَحَمَلَهَا الْإِنسَانُ ۖ إِنَّهُ كَانَ ظَلُومًا جَهُولًا

"We offered the Trust to the heavens, the earth, and the mountains, yet they refused to undertake it and were afraid of it; mankind undertook it- they have always been inept and foolish."

An agent that can modify a codebase bears a trust. The verse's structure is decisive: the heavens refused the trust, recognizing its weight; the human bore it and was called *ẓalūman jahūlā* (given to wrongdoing and ignorance). The design is therefore dual: (a) the agent operates within explicit authorization — it may not exceed asked scope (M5, anti-over-engineering); (b) the architecture *assumes the agent will err* and builds in rollback, sandboxing, and incremental commit as structural safeguards. Ibn Kathir's gloss (Ibn Abbās) ties amāna to accountability — outcome-linked feedback: actions must be traceable to outcomes, and outcomes feed the learning store. This is the governance boundary around Figures 1 and 2, realized concretely as fail-open safety (T2), reversibility, and auditable hooks.

9.7 The four governing concepts

Beyond the verses, four Qur'anic concepts structure the whole architecture, each load-bearing:

- **'ilm → fahm → ḥikma** (knowledge → understanding → wisdom) — the epistemological hierarchy that mandates a *layered* memory: raw logs ('ilm) at the base, a consolidation pass extracting patterns (fahm), a decision-support layer applying them (ḥikma). Dumping everything into one flat

vector store collapses the hierarchy — this is the architectural argument for STATE vs DECISIONS vs ARCHITECTURE as distinct stores with distinct update rules, not one file.

- **ḥifẓ + murāja'a** (preservation + spaced review) — memory is not write-once: a maintenance cycle reinforces recurring patterns, decays stale entries, and resolves contradictions. This is the memory-scoring rule (importance × recency-decay) and the validity-anchoring of Def. 4.
- **tabayyun** (verification before action) — the categorical gate of §9.2, the hard factor of Theorem D.
- **tadabbur** (structured reflection on consequences; root d-b-r, 'what follows') — the metacognitive controller as a trace-*forward* through the dependency graph: what comes after this change, what breaks, what it assumes. This is A1's consequence simulation, not a vague 'think again'.

> **The lens in one line**
>
> Every faculty the model lacks corresponds to an obligation the tradition names: *lā taqfu* (do not act without knowledge) → impact-closure; *tabayyun* (verify the report) → the gate; *ta'līm al-asmā'* (know the names) → the world-model; *al-qalam* (the pen) → persistent memory; *rabbi zidnī 'ilmā* (increase me in knowledge) → continual learning; *al-amāna* (the trust) → the stewardship boundary. The engineering says *how*; the lens says *why it is owed*.

---

10 The prototypes — instantiating the deterministic layer

Theory earns its keep when it runs. Two prototypes instantiate the deterministic layer's two hard functions — the impact closure (A1) and the routing/gate pair (A7 + A6) — and both are evaluated with the honesty the governing discipline demands: a perfect score on a self-built set demonstrates that a rubric *can separate* cases, never that it is a field benchmark.

10.1 Prototype I — the impact oracle (A1 made real)

An AST parser builds the typed artifact graph over a Python codebase; reverse reachability with confidence decay computes a file's blast radius — the closure Δ^* of Def. 3. Evaluated by mutation against two baselines:

Method	Precision	Recall	F1	Reading
edited-file only	1.00	0.53	0.65	never over-warns, misses half the blast radius
grep baseline	0.73	0.94	0.79	strong, but can still miss
graph oracle	0.63	**1.00**	0.75	**perfect recall — never misses an impacted file**

The oracle does not win on F1 — and the paper says so. Its distinguishing property is **perfect recall**, which is the safety property for the question "what will my edit break?": a reverse-reachability closure run to fixpoint *cannot miss a reachable dependent* (Thm. T5). It trades precision for that guarantee, tunable by the confidence threshold (best F1 = 0.79 at threshold 0.4). It scales: 303-node stdlib module in 18 ms, 1903-node module in 91 ms. This is anchor identity #1 confirmed on real code.

10.2 Prototype II — the complexity-router and assumption-gate (A7 + A6 + M1 + M2)

A transparent, feature-based rubric routes each task to a model tier (cheap/mid/premium) and gates under-specified requests before they reach a model at all — the assumption gate that answers the user's "biggest problem is assumption." The routing decision explains itself; it does *not* ask another opaque model, because that would reintroduce the `(1−p)` the deterministic layer exists to remove. Evaluated live on real models (haiku/sonnet/opus):

Metric	Result
Gate accuracy (should-ask, 30 tasks)	**30/30**, precision = recall = 1.00
Routing (well-specified tasks)	**21/21** exact tier

Real cost saved vs always-premium	**62.1%** on measured tokens
Execution-verified routed-down outputs	**3/3** passed real test cases

> **The honest caveat, stated wherever the numbers appear**
>
> The 30-task set is hand-labelled and the thresholds were tuned against it, so these numbers show the rubric *can separate* the cases — they are not a field benchmark. The cost figures, by contrast, are exact arithmetic on *real measured token counts* from live calls, and the correctness sub-experiment actually executed the cheaper models' code against test cases (the honesty core: routing down is only a saving if the cheap output is correct). Both prototypes ship as runnable packages with passing test suites.

---

11 forgekit — the deployed binding

The theory is implementation-independent; *forgekit* is one binding of it, and its precursor *claude-e2e-kit* is the reference realization on Claude Code. The mapping is exact:

Framework object	forgekit / claude-e2e-kit	Faculty / algorithm
Π_1 persistent store	`CLAUDE.md`, `docs/STATE.md`, `docs/DECISIONS.md`, `docs/ARCHITECTURE.md`	memory
Π_2 interception	`session-context.sh` (SessionStart), `docs-guard.sh` (Stop), `intent-router.sh` (UserPromptSubmit)	A5, A6, routing
Π_3 instructions	`CLAUDE.md` DoD + No-assumptions + Reflexion + effort-routing; `.claude/rules/`	I1–I4, M1–M6
Δ^* construction (A1)	`/impact` skill + `documentation-map.md`	impact-awareness
		self-correction

Verify operator (A3)	`/sync-docs` + `doc-sync` agent (project memory)	
Handoff (A4) / rehydrate (A5)	`/handoff` → STATE; SessionStart injection + `/catchup`	memory
Independent judge	`reviewer` agent (fresh context, adversarial, criteria-based)	self-correction
Phase gates	`sdlc-pilot` 7-phase skill (SPEC… MAINTAIN)	decomposition, goal-anchoring

The kit grounds its mechanisms in the same literature the substrate's faculties cite — CoALA's four memories, MemGPT's paging, Generative Agents' recency×importance scoring, Reflexion's verbal reinforcement, ReAct's reason–act interleaving, LLM-as-Judge's independent evaluator, AlphaCodium's phase-gated flow, Voyager's growing skill library, and the classical decision loops (BDI, OODA/PDCA, HTN). It is the same architecture, cited from the same shelf, and shipped. The two prototypes of §10 slot in as the mechanical cores of `/impact` and the effort-router.

> **Why one binding matters for the theory**
>
> That an independent team, starting from production failures rather than from cognitive faculties, built the same seven algorithms and stated the same design law is the strongest available evidence that the architecture is *discovered, not invented* — a convergent solution to a structural problem, the way distributed systems converge on snapshot+WAL. The synthesis does not merge two guesses; it records a convergence.

---

12 Honest limits — what no architecture can guarantee

- **Semantic correctness is undecidable.** The gate proves "a doc artifact changed", not "the docs are now true"; A3's grep proves "mentions were visited", not "the prose is accurate". By Rice's theorem the last layer is

unavoidable — tests for behaviour, human review for meaning. The architecture's job is to make that review *cheap*: everything arrives already-attempted, with an updated / verified / doubts report.

- **The soft layer's p is real but bounded.** Theorem D quantifies the residual; it does not abolish it. A determined agent can satisfy the letter of a gate (touch STATE with one line) without its spirit — which is why no single layer is trusted, by design.
- **$R_{\text{discovered}}$ misses unnamed coupling.** A doc that describes behaviour without naming any identifier has no grep edge; such couplings must be lifted into R_{declared} (the documentation map) — exactly what that table is for.
- **One prototype is not five faculties.** The substrate prototypes instantiate impact-awareness and routing/gating well; memory, learning, and imagination remain the harder research frontier, and the honest ecosystem map marks the assumption gate (M2) and outcome-based learning as genuine whitespace the current stack does not fill.
- **The lens is framing, not proof.** Every Qur'anic mapping motivates a design choice; none of them *validate* one. The engineering in §4–§7 stands on its own or not at all — the tradition tells us why a safeguard is owed, never that it works.

---

13 Conclusion

A language model that writes code is a fixed probabilistic map, and three independent efforts — one from cognition, one from production failures, one from a shipped codebase — converged on the same remedy: wrap it in an external, stateful architecture that supplies the faculties it structurally lacks. This paper showed they describe one object. The impact-awareness faculty *is* the change-closure fixpoint; the assumption gate *is* the amnesia equation; and both rest on a single theorem — reliability is the product of a *probabilistic* instruction layer that raises $p<1$ and a *deterministic* interception layer that guarantees a floor, with neither alone sufficient.

That theorem is the formal content of a plain discipline: *the output of a probability engine is never to be trusted on its own; trust is earned by an external check.*

The Qur'anic lens gives that discipline its oldest names — *lā taqfu*, do not pursue what you do not know; *tabayyun*, verify the report before you act; *al-amāna*, the weight of a trust accepted by one who may err. The mathematics says how to build the check. The tradition says why it is owed. The codebase shows it runs.

Companion artifacts: the three-way crosswalk (JSON + markdown), the graded reference set (Appendix A), and two runnable prototype packages (impact-oracle, router-gate). This synthesis consolidates and does not supersede the v2 *Theory → Evidence → Build-Map* edition, which carries the empirical evidence layer and the full ecosystem map.

Appendix A — Graded reference set (new sources)

The synthesis draws in a body of cognitive-architecture and process literature beyond the substrate paper's original 32 references. Each new source was independently verified this pass — modern arXiv sources by direct metadata fetch, classical works by primary-host search or established secondary knowledge — and graded: **confirmed** (record retrieved, attribution matches), **traceable** (the work clearly exists and is correctly attributed, but rests on established secondary knowledge rather than a single retrievable record), **unverifiable** (could not confirm). The tally: **8 confirmed, 6 traceable, 0 unverifiable.**

Source	ID	Grade	Note
Cognitive Architectures for Language Agents Theodore R. Sumers, Shunyu Yao, Karthik Narasi, 2023	`2309.02427`	confirmed	Retrieved via arXiv metadata API; title/authors match claim exactly. Unifies memory, planning/reasoning, action, and learning modules into a single CoALA framework for language agents, giving the cognitive-substrate work's memory/im…
ReAct: Synergizing Reasoning and Acting in Language	`2210.03629`	confirmed	Retrieved via arXiv metadata API; title/authors match claim exactly. Interleaves reasoning traces with actions in a single

Shunyu Yao, Jeffrey Zhao, Dian Yu et al., 2022			LLM prompt loop, the foundational pattern the cognitive-substrate and reliability-framework agent loops build on.
A Survey on LLM-as-a-Judge Jiawei Gu, Xuhui Jiang, Zhichao Shi et al., 2024	`2411.15594`	confirmed	Retrieved via arXiv metadata API; title/authors match claim exactly. Surveys the emerging practice of using LLMs themselves as evaluators/judges, directly relevant to any impact-awareness or self-correction mechanism that relies on an LL…
Code Generation with AlphaCodium: From Prompt Engine Tal Ridnik, Dedy Kredo, Itamar Friedman, 2024	`2401.08500`	confirmed	Retrieved via arXiv metadata API; title/authors match claim exactly. Replaces single-shot prompting with an iterative 'flow engineering' test-generate-fix loop for code generation, an applied precedent for the reliability frame…
SWE-agent: Agent-Computer Interfaces Enable Automate John Yang, Carlos E. Jimenez, Alexander Wettig, 2024	`2405.15793`	confirmed	Retrieved via arXiv metadata API; title/authors match claim exactly. Introduces an agent-computer interface (a constrained action/observation space) purpose-built for LM agents doing software engineering, directly relevant to forgekit&#x…
Voyager: An Open-Ended Embodied Agent with Large Lan Guanzhi Wang, Yuqi Xie, Yunfan Jiang et al., 2023	`2305.16291`	confirmed	Retrieved via arXiv metadata API; title/authors match claim exactly. Demonstrates a lifelong-learning embodied agent that maintains and grows a skill library over time, the clearest

			existing analogue to persistent, growing external memor…
Agentic AI in the Software Development Lifecycle: Ar Happy Bhati, 2026	2604.26275	confirmed	Retrieved via arXiv metadata API; the ID resolves to a REAL paper dated 2026-04-29 (April 2026, which is in the past relative to today, 2026-07-11 — so 'future-dated' only relative to the cited work's original claim date, …
Agent-as-a-Judge Runyang You, Hongru Cai, Caiqi Zhang et al., 2026	2601.05111	confirmed	Retrieved via arXiv metadata API; ID resolves to a REAL paper dated 2026-01-08. Title is exactly 'Agent-as-a-Judge' and abstract frames it as 'the first comprehensive survey' of the Agent-as-a-Judge paradigm, matching…
Agent-as-a-Judge: Evaluate Agents with Agents Zhuge, Zhao, Ashley, Wang, Khizbullin, Xiong, , 2024	2410.10934	confirmed	The founding Agent-as-a-Judge paper that coined the term; added on the verification track's explicit recommendation to disambiguate from the 2026 survey (2601.05111).
HTN Planning: Complexity and Expressivity Erol, Hendler, Nau, 1994	–	traceable	Classical AI-planning paper, no arXiv/OpenAlex record. Verified via live web_search (this turn) against AAAI's own paper page, Semantic Scholar, and a downstream paper's reference list (arXiv:1403.7426, 'An Overview of Hie…
BDI Agents: From Theory to Practice Rao, Georgeff, 1995	–	traceable	Classical agent-architecture paper, no DOI/arXiv record. Verified via live web_search (this turn) against the AAAI-hosted

			PDF (cdn.aaai.org/ICMAS/1995/ICMAS95-042.pdf), gabormelli.com/RKB, and multiple independent downstream reference li…
Thinking, Fast and Slow Kahneman, D., 2011	–	traceable	Classical trade/academic book, not indexed on arXiv or as a journal article with a DOI in the usual sense; existence and content (System 1 / System 2 dual-process framing) are well-established general knowledge, not independently re-veri…
Über das Gedächtnis: Untersuchungen zur experimentel Ebbinghaus, H., 1885	–	traceable	Foundational 1885 monograph establishing the forgetting curve; predates modern indexing entirely, existence is well-established historical/secondary knowledge, not independently re-verified against a bibliographic API in this pass.
OODA Loop (Observe-Orient-Decide-Act) Boyd, J.R., None	–	traceable	No formal published paper exists — Boyd never formally published the OODA loop in a journal; it survives via briefing-slide decks and secondary military-strategy literature. Rubric correctly flags this as having 'no formal paper.&#x…
PDCA / Plan-Do-Check-Act cycle (the 'Shewhart Cycle' Shewhart, W.A. (originator); Deming, W.E. (pop, None	–	traceable	Management/quality-control doctrine spanning multiple books across decades, not a single citable paper. Existence and attribution (Shewhart origin, Deming popularization) are well-established secondary knowledge, not independently re-ver…

The two future-dated identifiers — verified, not assumed

Two IDs in the source material are dated 2026. Both *resolve to real preprints* whose content matches the claim, confirmed by direct arXiv fetch rather than by topic plausibility. `arXiv:2604.26275` ("Agentic SDLC", Apr 2026) is a genuine but *single-author, non-peer-reviewed* preprint — cited as a recent preprint claim, not an established result. `arXiv:2601.05111` ("Agent-as-a-Judge", Jan 2026) is a real survey, but is a *different paper* from the founding work that coined the term — Zhuge et al. 2024 (`arXiv:2410.10934`), which is the reference this paper uses for the independent-judge concept in §11. Both are recorded here so the distinction is not lost.

References

Memory & persistence

1. **Weston, Chopra, Bordes** (2014). Memory Networks. arXiv:1410.3916.
2. **Sukhbaatar, Szlam, Weston, Fergus** (2015). End-To-End Memory Networks. arXiv:1503.08895.
3. **Graves, Wayne, Danihelka** (2014). Neural Turing Machines. arXiv:1410.5401.
4. **Graves et al.** (2016). Hybrid computing using a neural network with dynamic external memory. Nature.
5. **Lewis et al.** (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401 / NeurIPS.
6. **Packer et al.** (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
7. **Park et al.** (2023). Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442.
8. **McClelland, McNaughton, O'Reilly** (1995). Why there are complementary learning systems in the hippocampus and neocortex. Psychological Review.
9. **Kumaran, Hassabis, McClelland** (2016). What Learning Systems do Intelligent Agents Need? Complementary Learning Systems Theory Updated. Trends in Cognitive Sciences, 10.1016/j.tics.2016.05.004.
10. **Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan et al.** (2023). Cognitive Architectures for Language Agents. arXiv:2309.02427.

11. **Ebbinghaus, H.** (1885). Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie (Memory: A Contribution to Experimental Psychology). Leipzig: Duncker & Humblot (book/monograph).

Learning without weight updates

1. **Kirkpatrick et al.** (2017). Overcoming catastrophic forgetting in neural networks. PNAS / arXiv:1612.00796.
2. **Sun et al.** (2020). Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. arXiv:1909.13231 / ICML 2020.
3. **Ba, Hinton, Mnih, Leibo, Ionescu** (2016). Using Fast Weights to Attend to the Recent Past. arXiv:1610.06258.
4. **Hu et al.** (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
5. **von Oswald et al.** (2022). Transformers Learn In-Context by Gradient Descent. arXiv:2212.07677.
6. **Dai et al.** (2022). Why Can GPT Learn In-Context? Language Models Implicitly Perform Gradient Descent as Meta-Optimizers. arXiv:2212.10559.
7. **Guanzhi Wang, Yuqi Xie, Yunfan Jiang et al.** (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

Reasoning, verification & judgement

1. **Cobbe et al.** (2021). Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.
2. **Lightman et al.** (2023). Let's Verify Step by Step. arXiv:2305.20050.
3. **Shinn et al.** (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.
4. **Madaan et al.** (2023). Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651.
5. **Wang et al.** (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171.
6. **Huang et al.** (2023). Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798.
7. **Shunyu Yao, Jeffrey Zhao, Dian Yu et al.** (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
8. **Jiawei Gu, Xuhui Jiang, Zhichao Shi et al.** (2024). A Survey on LLM-as-a-Judge. arXiv:2411.15594.
9. **Zhuge, Zhao, Ashley, Wang, Khizbullin, Xiong, Liu, Chang, Krishnamoorthi, Tian, Shi, Chandra, Schmidhuber** (2024). Agent-as-a-Judge: Evaluate Agents with Agents. arXiv:2410.10934.

10. **Runyang You, Hongru Cai, Caiqi Zhang et al.** (2026). Agent-as-a-Judge. arXiv:2601.05111.
11. **Kahneman, D.** (2011). Thinking, Fast and Slow. Farrar, Straus and Giroux (book).

World-model & code structure

1. **Ha, Schmidhuber** (2018). World Models. arXiv:1803.10122.
2. **Schrittwieser et al.** (2020). Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. arXiv:1911.08265 / Nature.
3. **Hafner et al.** (2023). Mastering Diverse Domains through World Models. arXiv:2301.04104.
4. **Hafner et al.** (2019). Dream to Control: Learning Behaviors by Latent Imagination. arXiv:1912.01603.
5. **Friston** (2010). The free-energy principle: a unified brain theory?. Nature Reviews Neuroscience.
6. **Allamanis, Brockschmidt, Khademi** (2018). Learning to Represent Programs with Graphs. arXiv:1711.00740 / ICLR 2018.
7. **Yamaguchi, Golde, Arp, Rieck** (2014). Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE S&P 2014.
8. **Alon, Zilberstein, Levy, Yahav** (2019). code2vec: Learning Distributed Representations of Code. POPL 2019 / PACMPL.
9. **Horwitz, Reps, Binkley** (1990). Interprocedural Slicing Using Dependence Graphs. ACM TOPLAS.
10. **Tip** (1994). A Survey of Program Slicing Techniques. Journal of Programming Languages.
11. **Ferrante, Ottenstein, Warren** (1987). The Program Dependence Graph and Its Use in Optimization. ACM TOPLAS.

Agent process & software lifecycle

1. **Tal Ridnik, Dedy Kredo, Itamar Friedman** (2024). Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. arXiv:2401.08500.
2. **John Yang, Carlos E. Jimenez, Alexander Wettig et al.** (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793.
3. **Erol, Hendler, Nau** (1994). HTN Planning: Complexity and Expressivity. Proceedings of the 12th National Conference on Artificial Intelligence (AAAI-94), Vol. 2, pp. 1123-1128.
4. **Rao, Georgeff** (1995). BDI Agents: From Theory to Practice. Proceedings of the First International Conference on Multiagent Systems (ICMAS-95), pp. 312-319.

5. **Boyd, J.R.** (None). OODA Loop (Observe-Orient-Decide-Act). Unpublished briefings/lectures (e.g. 'A Discourse on Winning and Losing', unpublished collection of briefing slides, c. 1976-1996); no single peer-reviewed paper.
6. **Shewhart, W.A. (originator); Deming, W.E. (popularizer)** (None). PDCA / Plan-Do-Check-Act cycle (the 'Shewhart Cycle', popularized by Deming). Originates in Shewhart, Statistical Method from the Viewpoint of Quality Control (1939); popularized in Deming, Out of the Crisis (1982) and later works.
7. **Happy Bhati** (2026). Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence, and the Reshaping of Software Engineering. arXiv:2604.26275.

Appendix B — The crosswalk artifact

The full three-way crosswalk (§8) is provided as a machine-readable companion in two forms: `crosswalk.json` (structured, with the notation reconciliation and the three anchor identities) and `crosswalk.md` (readable table). Together with this paper, the two graded map files from the v2 edition (evidence map, ecosystem map), and the two runnable prototype packages (impact-oracle, router-gate), they form the complete synthesis deliverable set.
